

ADC-aware PTQ and Post-ADC Low-Rank Correction for Llama

Analog in-memory compute • Post-training quantization

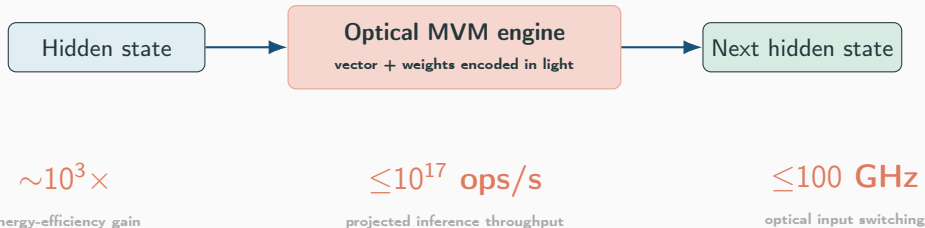
Alina Burykina

Constructor University | April 2026

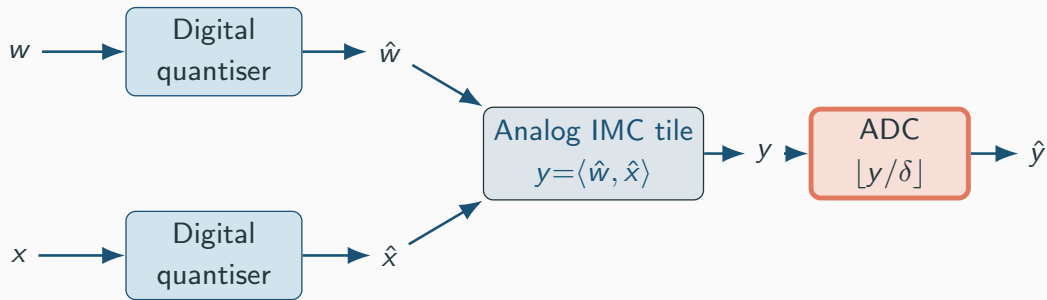
Optical GPU for LLM Inference

LLM inference is mostly dense linear algebra

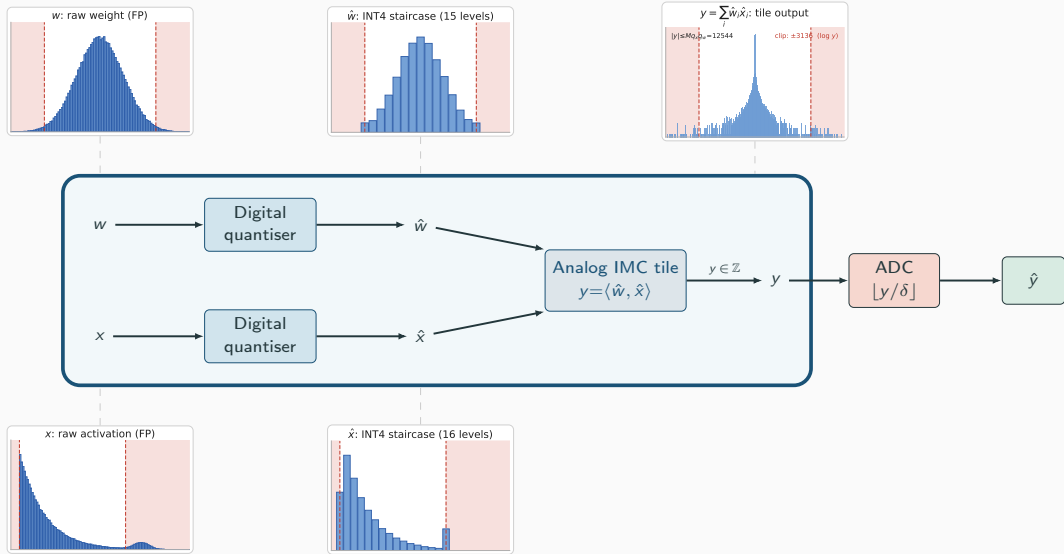
QKV projections · FFN · Output projection



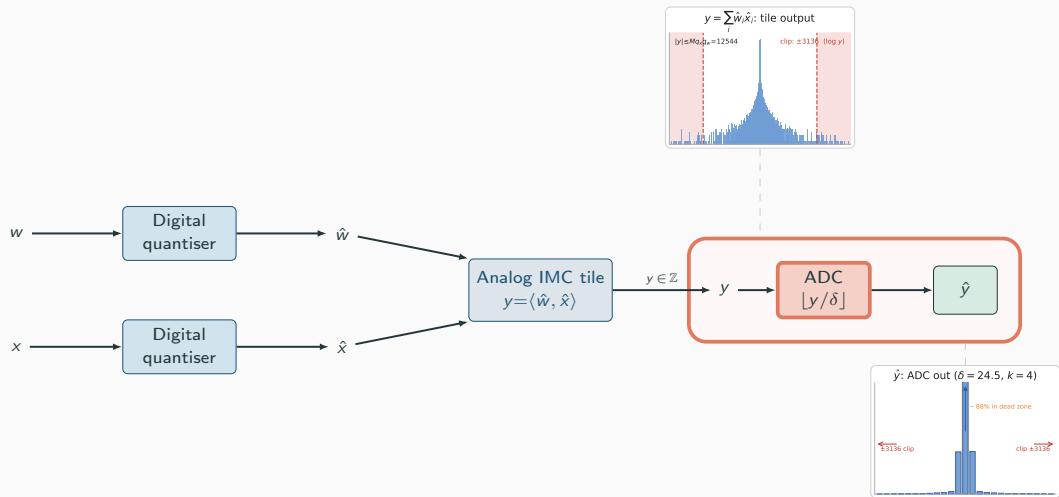
Analog MVM Signal Path



Digital Quantisation



ADC: Reading the Analog Accumulation



FlatQuant: Rotate, Flatten, Calibrate per Block

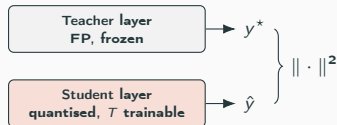
1. Linear invariance.

$$y = xW^{\top} = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^{\top})}_{\tilde{W}^{\top}}.$$

2. Kronecker factorisation.

$$T = T_1 \otimes T_2, \quad T_1, T_2 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}$$

3. Teacher-student block-wise calibration.



Q1 How severe is the ADC bottleneck on LLM inference?

→ *quantify the gap: FP vs bypass vs ADC*

Q2 Can ADC-aware PTQ alone recover FP quality?

→ *block-wise prop., α -mix, diag, staged training*

Q3 When PTQ plateaus, what is the minimum intervention beyond it?

→ *Post-ADC residual LoRA*

Q4 Does the correction generalise out-of-domain?

→ *train LoRA on WikiText-2, evaluate on C4*

Block-wise Calibration (FlatQuant Baseline)

FlatQuant standard. Train one block at a time, both teacher and student see the **same FP input** x^* :

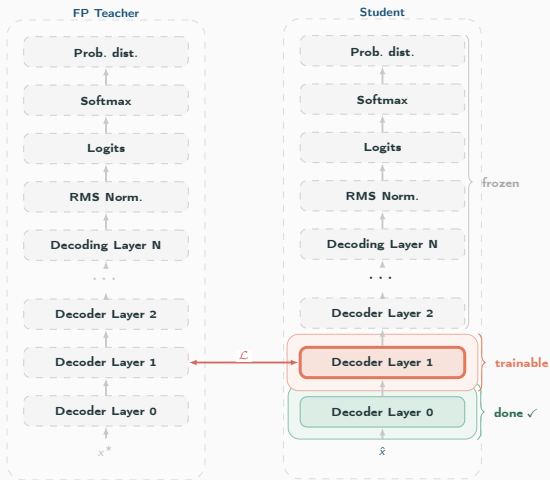
$$\mathcal{L} = \|\hat{B}(x^*) - B_{\text{FP}}(x^*)\|_2^2$$



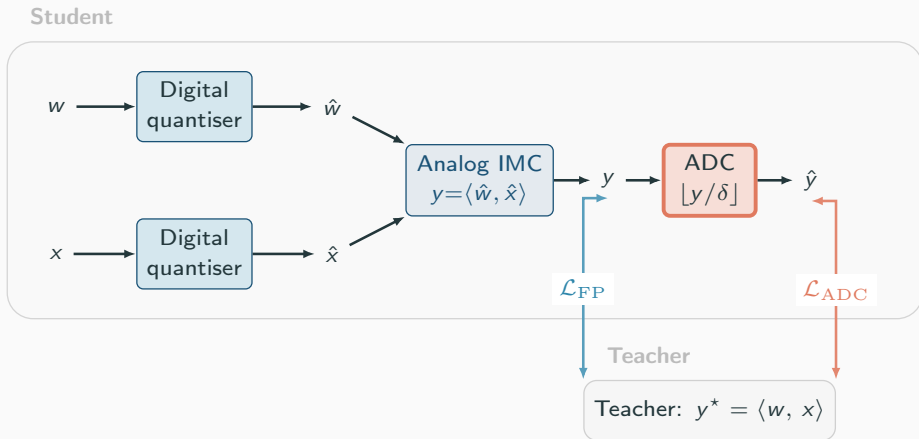
Our Extension I: Cross-block Propagation

Departure from FlatQuant. Input to block ℓ is the **ADC-quantised output of block $\ell-1$** , not the clean FP input.

Error accumulates the same way at train and inference.



Our Extension I: α -Mixed Objective



$$\mathcal{L}(\alpha) = (1-\alpha) \mathcal{L}_{\text{FP}} + \alpha \mathcal{L}_{\text{ADC}}$$

Trainable Diagonals (FlatQuant Component)

Absorb a trainable diagonal $D = \text{diag}(d_1, \dots, d_c)$ into the rotation:

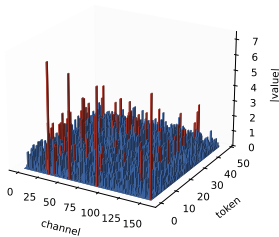
$$\tilde{T} = T \cdot D, \quad d_i > 0$$

Invariance preserved — opposite scaling on the next layer:

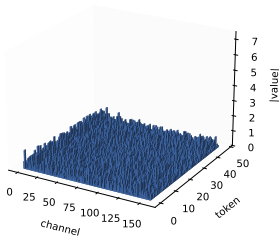
$$y = (x D^{-1} T^{-1}) (T D W^T).$$

FlatQuant diagonal: what it adds on top of the Kronecker rotation | Llama-3.2-1B layer 5.mlp.up_proj input | 5 ep × 16 samples

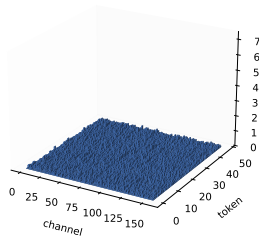
Original activation
(strong outliers)



After Kronecker rotation only
(outliers redistributed)



After Kronecker + diagonal
(flat across channels)



Our Extension II: Selective Diagonal Placement

We split FlatQuant's global diagonal by block type.

- `diag_attn`: before q/k/v/o projections
attention-head variance
- `diag_mlp`: before gate/up/down
SwiGLU outliers

Ablation (Llama-3.2-1B, w4a4, $\alpha=0.5$, ADC PPL↓):

α -mix base +	PPL _{bypass}	PPL _{ADC}
— (no diag)	24.24	31.22
diag_mlp only	19.37	31.65
diag_attn only	24.98	29.41
both diagonals	20.23	27.56

Staged Training: Why Two Stages?

MLP wants it clean

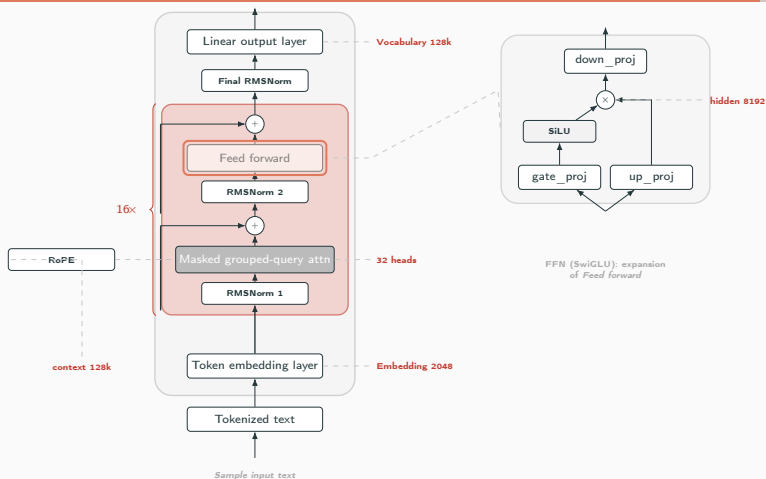
best $\text{PPL}_{\text{bypass}}$ with `diag_mlp`, $\alpha=0$ —
ADC suffers.

Attention wants ADC noise

best PPL_{ADC} with `diag_attn`, $\alpha \approx 0.5$ —
bypass suffers.

Train sequentially — MLP first, then add attention.

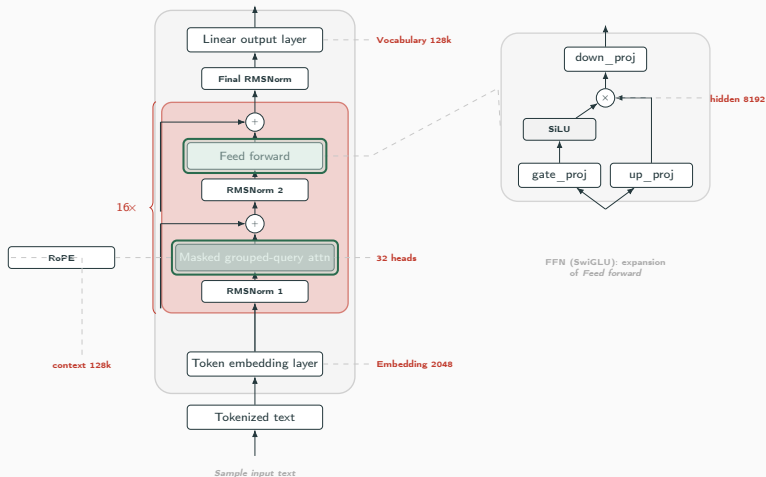
Staged Training: What Is Learnt, When



Stage A | MLP-first

Freeze attention. Train $T_{g,u,d}$ and $D_{g,u}^m$ with $\alpha=0$ — clean warmup.

Staged Training: What Is Learnt, When



Stage B | full block

Unfreeze attention. Add

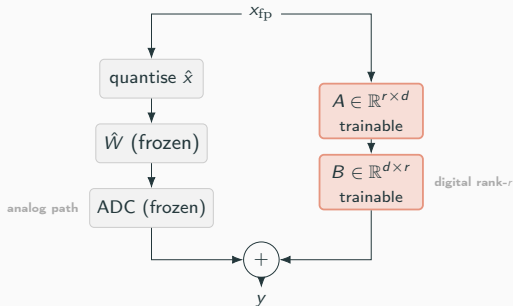
$T_{q,k,v,o}$ and $D_{q,k,v}^a$. Turn on $\alpha=0.5$.

PTQ Results: INT8 and INT4 ADC, WikiText-2

Method	bits (w / a)	PPL _{bypass}	PPL _{ADC}
FP baseline	16 / 16	8.68	8.68
FlatQuant per-proj MSE	8 / 8	10.01	28.86
+ propagated calib (block)	8 / 8	11.00	14.41
+ staged + diag (full pipeline)	8 / 8	19.32	16.85
FlatQuant per-proj MSE	4 / 4	15.41	2354
+ 1024 calib samples	4 / 4	19.80	56.83
+ block prop., $\alpha=0.5$	4 / 4	24.24	31.22
+ diag (bounded LET)	4 / 4	20.23	27.56
+ staged: MLP $\rightarrow$ attn	4 / 4	17.83	26.66

Method II: Post-ADC Residual LoRA

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_{\text{LoRA}}}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r}$$



Loss: distill from FP teacher.

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p} \| p_{\text{FP}})$$

What is trained:

- Only A, B for each of 7 projections ($q, k, v, o, \text{gate}, \text{up}, \text{down}$).

LoRA Ablations and Overhead

Ablations (Llama-3.2-1B, INT4, $\text{PPL}_{\text{ADC}} \downarrow$)

Base PTQ: staged INT4 ($\text{PPL}_{\text{ADC}} \approx 27.5$).

Axis	Setting	PPL_{ADC}
Rank r	$r=1$	15.90
	$r=4$	15.57
	$r=8$	15.24
Loss	CE only	15.57
	CE + KL	14.33
Layers	last 8 only	20.08
	first 8	16.64
	all 16	15.57
Placement	Pre-ADC	105.6
	Post-ADC	15.57
Coverage	down + o (CE + KL)	14.33
	all 7 (CE + KL)	14.03

Overhead on Llama-3.2-1B

($d=2048$, 16 blocks, $r=4$, all 7 proj.)

total LoRA params	~ 2.8 M
as fraction of FP weights	0.22%
FP-teacher forward	$1\times$ cached
student forward + back	$\sim 5\times$ FP MAC
training (1 GPU, 5 ep.)	~ 45 min
inference MAC overhead	$< 0.5\%$

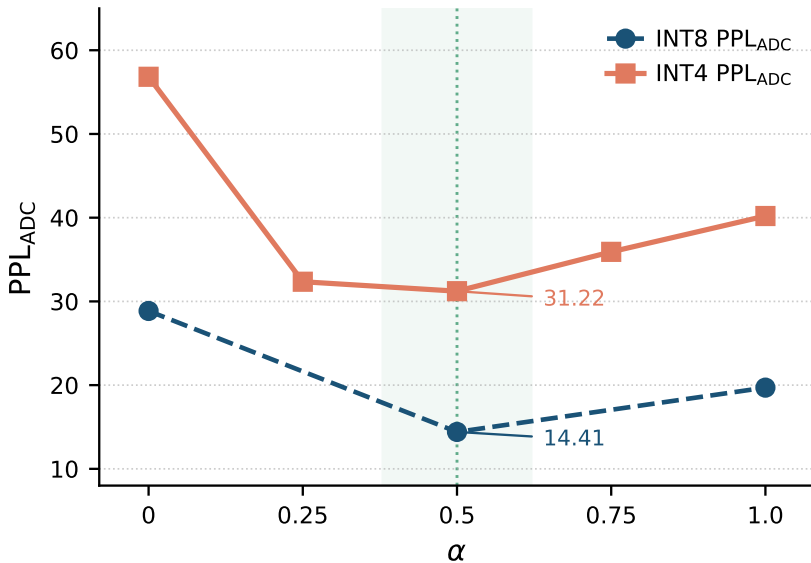
Final Results: INT4+LoRA Beats INT8 PTQ

Model	Method	Wiki PPL	C4 PPL	HellaSwag	MMLU	WinoGrande
Llama-3.2-1B	FP16	8.68	13.13	64.19	32.15	63.06
	INT8 PTQ	19.32	28.08	57.07	26.78	57.06
	INT8+ADC PTQ	16.85	29.56	45.04	26.83	56.59
	INT4 PTQ	17.83	28.58	50.72	23.68	55.96
	INT4+ADC PTQ	26.66	45.62	40.03	24.41	53.43
	INT4+ADC+LoRA	14.03	23.20	49.96	25.72	53.67

Takeaways

1. **First adaptation of FlatQuant to analog ADC.** Hardware floor quantisation (fixed δ) integrated into block-wise calibration.
2. **Cross-block α -mixed propagation is the biggest PTQ lever.** On top of FlatQuant's block-wise objective, halves INT8 ADC PPL (28.86 $\rightarrow$ 14.41); enables INT4 recovery.
3. **Selective diagonal placement.** MLP-first / attn-second staged training; `down_trans` is the key driver inside MLP.
4. **Post-ADC LoRA breaks the PTQ plateau.** 0.22% extra params $\rightarrow$ INT4 (14.03) matches INT8 PTQ (16.85). Generalises to C4 (45.6 $\rightarrow$ 23.2 ADC PPL, -50%).

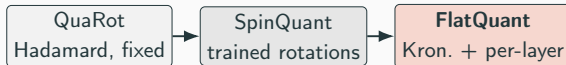
Backup: α -Sweep — $\alpha=0.5$ Wins



Backup: Rotation Line of Work

The rotation line of work.

Insert an orthogonal T so that $WT^{-1} \cdot Tx = Wx$ and Tx is easier to quantise:

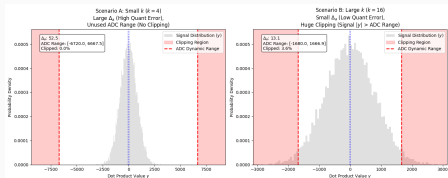


Why FlatQuant for our work:

- *trainable* — we can add objectives (ADC-aware);
- *Kronecker* ($T = T_1 \otimes T_2$) — memory & compute $\mathcal{O}(\sqrt{d})$;
- per-projection rotations fold naturally into block-wise calibration.

Why ADC is Hard for LLMs: the Outlier Dilemma

LLM activations are heavy-tailed. A tiny fraction of channels carry magnitudes 10–100 $\times$ larger than the rest (Dettmers 2022; Xiao 2023).



Llama-3.2-1B q_proj dot-product distribution, two ADC step sizes

Why we can't just pick a good k :

Large k (fine δ)

outliers saturate the ADC range
 $\Rightarrow$ heavy clipping
model loses critical channels
good ADC SNR on bulk

bad downstream accuracy

Small k (coarse δ)

outliers stay in range
dynamic range wasted on outliers
bulk collapses into *dead zone*

bad ADC SNR everywhere

Takeaway

Hardware-fixed δ + heavy-tailed activations $\Rightarrow$ **no single k works**. The algorithm must reshape x and w *before* they hit the array so that y lands in the ADC's usable range.

Backup: Complete INT8 Results (v2)

Setup: Llama-3.2-1B, w8a8, 128 calibration samples. Testing two ideas on top of baseline FlatQuant: (1) *bin-centre loss* — pushes y_{int} toward ADC bin centres during PTQ; (2) *propagated calibration* — each block sees ADC-quantised inputs from the previous block, matching inference conditions. Key finding: propagation gives a 2× ADC PPL drop; bin-centre loss alone gives nothing.

Experiment	Description	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	dead_rate
baseline	FlatQuant w8a8	10.01	28.86	10.3%
center mid	bin-centre $\lambda = 0.1$	10.01	28.86	10.3%
prop	propagated calibration	11.01	14.55	10.3%
prop+center	propagated + bin-centre	11.00	14.41	10.3%

Backup: Complete INT4 v2 Alpha-Sweep

Setup: Llama-3.2-1B, w4a4, 1024 samples. Sweeping α in the mixed loss $\mathcal{L}(\alpha) = (1-\alpha)\mathcal{L}_{\text{FP}} + \alpha\mathcal{L}_{\text{ADC}}$. $\alpha=0$ = clean FP inputs only; $\alpha=1$ = ADC-noisy inputs only. Gap = ADC PPL / bypass PPL. Last two rows add diagonals and staged training on top of $\alpha=0.5$.

Experiment	α	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	gap
baseline_1024	0.0	19.80	56.83	$\times 2.87$
prop_full_1024	1.0	30.32	40.20	$\times 1.32$
propalpha_075_1024	0.75	27.46	35.92	$\times 1.31$
propalpha_05_1024	0.5	24.24	31.22	$\times 1.29$
propalpha_025_1024	0.25	21.65	32.35	$\times 1.49$
add_diag + $\alpha=0.5$	0.5	20.23	27.56	$\times 1.36$
staged_mlp_attn	0.5	18.50	27.60	$\times 1.49$

Backup: Complete v7 LoRA Ablation

Setup: Post-ADC LoRA correction on frozen PTQ model (Llama-3.2-1B, w4a4, staged PTQ base: bypass=17.89, ADC=27.55). LoRA adapters trained with CE loss on wikitext2, optionally + KL divergence term. Ablating: target projections (down+o vs all 7), rank (1–8), loss (CE vs CE+KL), layer selection (first/last 8), and pre- vs post-ADC placement (pre-ADC fails catastrophically).

Experiment	Targets	Loss	Rank	PPL _{bypass}	PPL _{ADC}
base (no LoRA)	—	—	—	17.89	27.55
r1_down_o	down+o	CE	1	—	15.90
r2_down_o	down+o	CE	2	—	15.60
r4_down_o	down+o	CE	4	18.68	15.57
r8_down_o	down+o	CE	8	—	15.24
r4_down_o_ce_kl	down+o	CE+KL	4	19.54	14.33
r4_down_o_first8	down+o	CE	4	18.42	16.64
r4_down_o_last8	down+o	CE	4	18.62	20.08
pre_adc_r4_down_o	down+o	CE	4	123.34	105.63
r4_all_ce	all 7	CE	4	17.62	16.68
r4_all_ce_kl	all 7	CE+KL	4	19.13	14.03

Backup: v8 Generalisation to C4

Setup: Same PTQ base as v7, LoRA trained on wikitext2 only, evaluated on both wikitext2 and C4. Checks whether the best v7 config (r4_all_ce_kl) generalises out-of-domain. Without LoRA, the base model has a large C4 ADC gap (46.40 vs bypass 29.41); LoRA closes it to 23.30 on C4 — confirming the correction is not overfitting to the training distribution.

Experiment	Targets	Wiki bypass	Wiki ADC	C4 bypass	C4 ADC
base (no LoRA)	—	18.49	27.26	29.41	46.40
rank8_down_o (CE)	down+o	18.51	15.47	30.48	29.39
r4_down_o_ce_kl	down+o	19.05	14.27	29.37	23.88
r4_all_ce_kl	all 7	18.70	13.96	29.10	23.30

Backup: v9 Data Efficiency (lora_nsamples sweep)

Setup: Best config (r4_all_ce_kl, mvm_limit=256), PTQ fixed at 1024 samples. Varying how many calibration sequences LoRA sees during fine-tuning (64 $\rightarrow$ 1024). Shows LoRA improves even with 64 samples, but ADC PPL keeps improving up to 1024 — more data helps consistently.

Experiment	nsamples	Wiki PPL _{bypass}	Wiki PPL _{ADC}	C4 PPL _{bypass}	C4 PPL _{ADC}
r4_all_ce_kl_n64	64	18.42	21.07	29.50	35.62
r4_all_ce_kl_n128	128	18.21	18.09	28.97	30.19
r4_all_ce_kl_n256	256	18.66	15.88	28.84	26.13
r4_all_ce_kl_n512	512	17.99	14.82	28.20	24.75
r4_all_ce_kl_n1024	1024	18.92	14.06	28.79	23.56

Config: r4_all_ce_kl, mvm_limit=256. PTQ fixed at 1024 samples.

Backup: v9 KL Weight $\times$ Temperature Sweep

Setup: Base r4_all_ce_k1 (1024 samples). KL term $\lambda_{\text{KL}} \cdot D_{\text{KL}}(\hat{p}_{\text{ADC}} \parallel \hat{p}_{\text{FP}})$ with softmax temperature T . Higher T softens logits before KL, making the teacher signal smoother. $\lambda_{\text{KL}}=0.5$, $T=1$ gives the best C4 ADC PPL (23.10); very high λ ($2.0/T=4$) destabilises training.

Experiment	λ_{KL}	T	Wiki PPL _{bypass}	Wiki PPL _{ADC}	C4 PPL _{bypass}	C4 PPL _{ADC}
r4_kl025_t1	0.25	1.0	18.77	13.84	29.16	23.23
r4_kl025_t2	0.25	2.0	19.15	14.07	29.03	23.33
r4_kl05_t1	0.50	1.0	18.92	13.79	29.69	23.10
r4_kl05_t2	0.50	2.0	18.43	14.05	28.54	23.37
r4_kl10_t2	1.00	2.0	18.90	13.94	29.47	23.17
r4_kl20_t4	2.00	4.0	failed			

Base: r4_all_ce_k1 (1024 samples). Best: $\lambda = 0.5$, $T = 1$ — C4 ADC 23.10.

Backup: Stochastic Propagation (v5)

Setup: Instead of a fixed α , sample α_t each batch — either Bernoulli($\{0, 1\}$) or Beta(2, 2) (concentrated around 0.5). Bernoulli with staged training gives the best bypass PPL ever seen (16.94) but a bad ADC gap, because FP-only batches teach nothing about ADC noise. Beta(2, 2) $\approx$ deterministic $\alpha=0.5$ in expectation and achieves the best ADC PPL (27.82).

Experiment	Mode	PPL _{bypass}	PPL _{ADC}	gap
stoch_bern (flat)	Bernoulli	19.25	35.38	$\times 1.84$
stoch_beta2 (flat)	Beta(2,2)	22.90	31.40	$\times 1.37$
staged + Bernoulli	Bernoulli	16.94	32.44	$\times 1.92$
staged + Beta(2,2)	Beta(2,2)	17.16	27.82	$\times 1.62$
deterministic $\alpha=0.5$	—	24.24	31.22	$\times 1.29$

Bernoulli gives best bypass but worst ADC gap (hard 0/1 switching provides no ADC signal on FP batches). Beta(2,2) $\approx$ deterministic $\alpha=0.5$ in expectation.

Backup: Hardware Config & Delta Formula

Parameter	Value
b_w (weight bits)	4 / 8
b_x (act. bits)	4 / 8
b_a (ADC bits)	8
mvm_limit	256 / 1024
k (ADC parallelism)	16
δ (mvm=256, INT8)	≈ 2016
δ (mvm=256, INT4)	≈ 6.12
δ (mvm=1024, INT8)	≈ 8064
FlatQuant epochs	30
FlatQuant LR	0.005
Cal. samples (INT4)	1024
Cal. samples (INT8)	128

Delta formula ($q = 2^{b-1} - 1$):

$$\delta = \frac{2 \cdot \text{tile_in} \cdot q_x \cdot q_w}{2^{b_a} \cdot k}$$

- INT8 ($q=127$, mvm=256): $\frac{2 \cdot 256 \cdot 127^2}{256 \cdot 16} \approx 2016$
- INT4 ($q=7$, mvm=256): $\frac{2 \cdot 256 \cdot 7^2}{256 \cdot 16} \approx 6.12$
- INT8, mvm=1024: $\frac{2 \cdot 1024 \cdot 127^2}{256 \cdot 16} \approx 8064$

INT8: useful ADC bins in $\pm \sigma_z \approx 15\text{--}30$ (empirical) — explains why INT8 PTQ alone plateaus.

This Work in Context

Method	Key idea	Gap / limit
FlatQuant	Learned Kronecker-decomposed orthogonal transforms per layer	FP loss only; no ADC signal
OmniQuant / LET	Learnable per-channel equivalent scaling (diagonal)	No ADC-aware objective
QEP / prop. calib.	Feed quantised activations into the next layer's calibration	INT8 only; no mixed-loss
PACT / QDrop	Stochastic / clamped quant-aware training	Full re-training required
This work	ADC-aware PTQ (α -mixed, staged diag) + post-ADC LoRA	—

Key novelty

No prior method directly targets the **post-ADC residual** — our LoRA correction is the first to do so.

Post-ADC LoRA: Parameter and Compute Overhead

Trainable parameter count ($r=4$, all 7 projections):

Projection	A	B
q_proj	4×2048	2048×4
k_proj	4×2048	256×4
v_proj	4×2048	256×4
o_proj	4×2048	2048×4
gate_proj	4×2048	8192×4
up_proj	4×2048	8192×4
down_proj	4×8192	2048×4

Inference overhead (per forward pass):

- Extra FLOPs per projection: $d_{\text{in}} \cdot r + r \cdot d_{\text{out}} \ll d_{\text{in}} \cdot d_{\text{out}}$
- Example: q_proj: $8192+8192$ vs. $4M$ ($<0.5\%$ overhead)
- Memory: 2.8M fp32 params $\approx$ **11 MB** vs. ~ 2 GB base model
- LoRA correction runs on *digital* hardware — does not increase analog area

Bottom line

0.22% extra params, $< 0.5\%$ extra FLOPs, correction entirely in digital post-processing. The trade-off is **strongly favourable**.

Thank you.

Questions?

Alina Burykina | Constructor University | April 2026